

- 1: Introdução
- 2: Objetivo do projeto
- 3: Arquitetura da aplicação
- 4 : Frontend
- 5: Backend
- 6: Autenticação
- 7: Banco de dados
- 8: Sistema de reviews
- 9:Sistema de membros
- 10:Docker
- 11:Tecnologias utilizadas
- 12:Testes realizados
- 13:Conclusão

UNIVERSIDADE VEIGA DE ALMEIDA

KEVEN DE SOUZA OLIVEIRA DO CARMO

INDIVIDUAL

TRASHEIRA VIOLENTA

Aplicação Web para Cadastro e Consulta de Reviews
de Filmes, Séries e Jogos

Trabalho apresentado à disciplina de [BACKEND],
como parte dos critérios de avaliação.

Rio de Janeiro

2026

1. INTRODUÇÃO

O projeto **Trasheira Violenta** consiste em uma aplicação web voltada para a apresentação e gerenciamento de reviews de filmes, séries e jogos.

O sistema foi desenvolvido utilizando uma arquitetura composta por **frontend, backend e banco de dados**, executados em containers por meio do Docker.

Além da visualização dos conteúdos, a aplicação permite o cadastro de usuários, autenticação por login, cadastro de novas reviews, edição e exclusão de registros e um sistema demonstrativo de assinatura de membros.

O objetivo principal do projeto foi aplicar na prática conceitos relacionados ao desenvolvimento de aplicações web, integração entre frontend e backend, utilização de banco de dados e execução dos serviços utilizando Docker.

2. OBJETIVO DO PROJETO

O objetivo do sistema é disponibilizar uma plataforma na qual usuários possam consultar conteúdos relacionados a filmes, séries e jogos e, após realizar login, utilizar recursos adicionais da aplicação.

Entre as funcionalidades implementadas estão:

- cadastro e login de usuários;
- autenticação utilizando token;
- listagem de reviews;
- cadastro de novas reviews;
- edição de reviews;
- exclusão de reviews;
- páginas separadas para filmes, séries e jogos;
- busca de conteúdos;
- sistema de assinatura Básico e Premium;
- identificação do plano do usuário após a assinatura;
- execução do frontend, backend e banco de dados utilizando Docker.

3. ARQUITETURA DA APLICAÇÃO

A aplicação foi dividida em três componentes principais:

USUÁRIO / NAVEGADOR



FRONTEND

HTML + Tailwind CSS

JavaScript + Nginx

Porta 8080



BACKEND

Node.js + Express

Porta 3000



BANCO DE DADOS

PostgreSQL

Porta 5432

O frontend é responsável pela interface apresentada ao usuário.

O backend recebe as requisições enviadas pelo frontend, realiza as regras da aplicação e se comunica com o banco de dados.

O PostgreSQL é responsável pelo armazenamento permanente das informações da aplicação.

Os três componentes são executados em containers separados e são organizados utilizando o **Docker Compose**.

4. FRONTEND

O frontend foi desenvolvido utilizando **HTML, JavaScript e Tailwind CSS**.

As páginas principais da aplicação são:

index.html

login.html

membro.html

filmes.html

series.html

jogos.html

sobre.html

A página inicial apresenta as reviews e permite que usuários autenticados cadastrem novos conteúdos.

As páginas de Filmes, Séries e Jogos organizam os conteúdos de acordo com cada categoria.

A página de login permite entrar em uma conta existente ou cadastrar um novo usuário.

A página Seja Membro apresenta os planos disponíveis e uma simulação de assinatura.

O frontend é disponibilizado através de um container utilizando o servidor **Nginx**.

Print para colocar aqui:

Figura 1 – Página inicial da aplicação

Coloca um print do seu index.html já logado, mostrando:

Olá, seu nome | ★ Premium | Sair

5. BACKEND

O backend da aplicação foi desenvolvido utilizando **Node.js** com o framework **Express**.

Ele é responsável por processar as requisições do frontend e realizar operações no banco de dados.

Entre as principais rotas implementadas estão:

POST /cadastro

POST /login

GET /reviews

POST /reviews

PUT /reviews/:id

DELETE /reviews/:id

POST /assinaturas

GET /minha-assinatura

As rotas de cadastro, edição e exclusão de reviews utilizam autenticação para impedir alterações sem um usuário logado.

O backend é executado na porta:

3000

```
backend-1 | Servidor rodando na porta 3000
```

6. AUTENTICAÇÃO

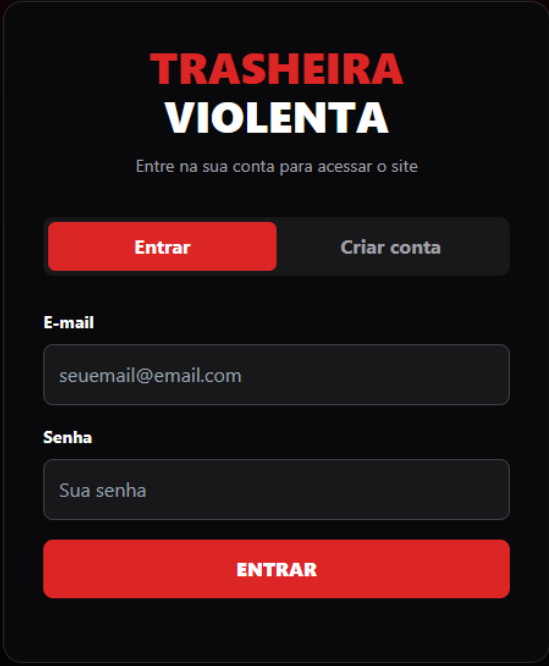
Para acessar as funcionalidades principais do sistema, o usuário precisa possuir uma conta.

Durante o cadastro, a senha é protegida utilizando a biblioteca **bcryptjs**, evitando seu armazenamento diretamente em texto simples.

Após o login ser realizado corretamente, o backend gera um **JWT (JSON Web Token)**.

Esse token é enviado pelo frontend nas operações que exigem autenticação.

Quando o usuário tenta cadastrar, editar ou excluir uma review, o backend verifica se o token recebido é válido antes de permitir a operação.



The image shows a login form for a website called "TRASHEIRA VIOLENTA". The form is centered on a dark background. At the top, the website name "TRASHEIRA VIOLENTA" is displayed in red and white. Below it, a subtitle reads "Entre na sua conta para acessar o site". There are two buttons: "Entrar" (red) and "Criar conta" (grey). Below these buttons are two input fields: "E-mail" with the placeholder "seuemail@email.com" and "Senha" with the placeholder "Sua senha". At the bottom of the form is a large red button labeled "ENTRAR".

7. BANCO DE DADOS

O banco utilizado no projeto foi o **PostgreSQL**.

As principais tabelas utilizadas são:

Tabela usuarios

Armazena os usuários cadastrados no sistema.

id

nome

email

senha

Tabela reviews

Armazena as avaliações cadastradas.

id

titulo

categoria

ano

nota

descricao

Tabela assinaturas

Armazena os planos associados aos usuários.

id

usuario_id

plano

valor

status

data_inicio

O relacionamento entre usuários e assinaturas é realizado através do campo:

usuario_id

8. SISTEMA DE REVIEWS

Após realizar login, o usuário consegue cadastrar uma nova review informando:

Título

Categoria

Ano

Nota

Descrição

Após o cadastro, os dados são enviados ao backend através de uma requisição HTTP e armazenados no PostgreSQL.

Também foram implementadas as funções de **edição e exclusão**, formando as principais operações de CRUD da aplicação.

CRUD representa:

Create – cadastrar

Read – consultar

Update – editar

Delete – excluir



O formulário, intitulado "CADASTRAR NOVA REVIEW", é apresentado em um fundo escuro. Ele contém cinco campos de entrada: um campo de texto para "Título", um menu suspenso para "Selecione a categoria", um campo de texto para "Ano", um campo de texto para "Nota de 0 a 10" e um campo de texto maior para "Descrição da review". Abaixo dos campos, há um botão vermelho com o texto "CADASTRAR REVIEW" em branco.

9. SISTEMA DE MEMBROS

A aplicação possui também um sistema demonstrativo de assinatura.

Foram criados dois planos:

Plano Básico – R\$ 9,90

Plano Premium – R\$ 19,90

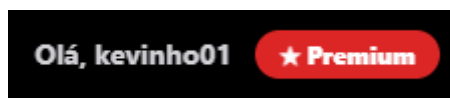
Quando o usuário confirma uma assinatura, o plano é associado à sua conta no banco de dados.

Após a assinatura, o sistema consulta o backend e apresenta o plano ao lado do nome do usuário.

Exemplo:

Olá, usuário ★ Premium Sair

A área de pagamento foi desenvolvida apenas para **demonstração acadêmica**, não realizando cobranças financeiras reais e não armazenando dados reais de cartão.



10. DOCKER

Para executar os diferentes componentes do sistema foi utilizado o **Docker**.

Foram criados três serviços:

frontend

backend

db

O serviço frontend executa o Nginx e disponibiliza as páginas da aplicação.

O serviço backend executa a API desenvolvida em Node.js.

O serviço db executa o PostgreSQL.

A inicialização dos serviços é realizada utilizando:

```
docker compose up --build
```

Também foi utilizado um volume para permitir a persistência dos dados do PostgreSQL, fazendo com que os registros permaneçam armazenados mesmo quando os containers são recriados.

11. TECNOLOGIAS UTILIZADAS

HTML

Tailwind CSS

JavaScript

Node.js

Express

PostgreSQL

Nginx

Docker

Docker Compose

JWT

bcryptjs

12. TESTES REALIZADOS

Durante o desenvolvimento foram realizados testes das principais funcionalidades da aplicação.

Foram verificados o cadastro e login de usuários, acesso autenticado, cadastro de reviews, consulta das informações armazenadas no banco, edição e exclusão das reviews, assinatura dos planos, exibição do plano Premium e funcionamento dos três containers.

Também foi verificada a persistência dos dados no PostgreSQL após a reinicialização dos containers.

13. CONCLUSÃO

O desenvolvimento do **Trasheira Violenta** permitiu integrar diferentes tecnologias em uma única aplicação.

O projeto possui frontend, backend e banco de dados funcionando de forma integrada, com cada componente executado em seu próprio container Docker.

Além da apresentação de conteúdos de filmes, séries e jogos, foram desenvolvidas funcionalidades de autenticação, gerenciamento de reviews e associação de planos aos usuários.

Dessa forma, o trabalho permitiu aplicar conceitos de desenvolvimento web, APIs, banco de dados, autenticação e containerização em uma aplicação funcional.